

Rapport d'analyse

Gestion de restaurant

Analyse basée sur le code présent dans le projet au 20 août 2026.

Ce document décrit le but du projet, son architecture, ses modèles, ses routes API, son authentification, ses permissions et ses flux principaux.

Important : ce rapport est descriptif. Aucun fichier source n'a été corrigé dans le cadre de cette analyse.

1. Présentation générale

gestion-de-restaurant est une API backend Node.js destinée à gérer l'activité d'un restaurant. Elle centralise les clients, les utilisateurs internes, les catégories, les plats, les commandes et les factures.

Les utilisateurs internes sont identifiés par des rôles : `admin`, `serveur` et `caissier`. Un rôle `client` existe également : il permet à un client de créer son compte, de passer ses propres commandes et de consulter ses propres factures.

L'application expose une API REST protégée par JWT et connectée à MongoDB.

2. Technologies

Technologie	Rôle
Node.js	Exécution du backend JavaScript.
Express	Serveur HTTP, routes et middleware.
MongoDB	Base de données.
Mongoose	Modèles, schémas et accès MongoDB.
bcrypt	Hashage et vérification des mots de passe.
jsonwebtoken	Création et vérification des tokens JWT.
Yup	Validation des données entrantes.
dotenv	Chargement des variables du fichier <code>.env</code> .

cors, helmet, morgan	Dépendances installées, mais non utilisées dans app.js.
swagger-jsdoc, swagger-ui-express	Dépendances installées, mais aucune documentation Swagger n'est configurée.
nodemon	Redémarrage automatique en développement.

3. Architecture

Les domaines principaux utilisent les couches route, controller, service, repository, model et validate. Le dossier authentication ajoute les middleware JWT, rôle et propriétaire.

```

Client HTTP
  ↓
Route Express
  ↓
authMiddleware
  ↓
roleMiddleware / ownerMiddleware
  ↓
validationMiddleware (Yup)
  ↓
Controller → Service → Repository → Model Mongoose → MongoDB

```

app.js configure Express et enregistre les routes. serveur.js écoute sur le port 8000.
commun/validator.js fournit le middleware Yup. config/db/db.config.js ouvre la connexion MongoDB.

4. Entités et relations

User

Compte d'accès au système : nom, prenom, email, telephone, mot_de_passe, role et id_client optionnel. Le mot de passe est exclu des lectures Mongoose par défaut.

Client

Consommateur du restaurant : matricule, nom, prenom, telephone, adresse et email.

Categori

Catégorie d'un plat : nom et description.

Plat

Article du menu : nom, description, prix, disponible et référence categori.

Commande

Commande d'un client enregistrée par un utilisateur. Elle contient id_user, id_client, des lignes de plats (id_plat, quantite, prixunitaire), date_commande, total et status.

Facture

Facture liée à un utilisateur, une commande et un client. Elle contient id_user, id_commande, id_client, montant_total, date_facture, statut et mode_paiement.

```
user.id_client → client
plat.categori → categori
commande.id_user → user
commande.id_client → client
commande.plat[].id_plat → plat
facture.id_user → user
facture.id_commande → commande
facture.id_client → client
```

5. Routes API

Authentification

Méthode	URL	Accès	Rôle
POST	/authentification/inscription-client	Créer client + compte	Public
POST	/authentification/connexion	Connexion et JWT	Public
GET	/authentification/moi	Utilisateur connecté	JWT

CRUD principaux

Entité	Routes	Accès principal
user	GET /, POST /creation, GET/PUT/DELETE /:id	Admin ; propriétaire pour son profil
client	GET /, POST /creation, GET/PUT/DELETE /:id	Admin, serveur, caissier selon l'action

plat	GET /, POST /creation, GET/PUT/DELETE /:id	Lecture personnel ; écriture admin
categori	GET /, POST /creation, GET/PUT/DELETE /:id	Lecture personnel ; écriture admin
commande	GET/POST/PUT/DELETE ; /mes-commandes ; /creation-client	Personnel selon rôle ; client pour ses données
facture	GET/POST/PUT/DELETE ; /mes-factures	Admin/caissier ; client pour ses factures

6. Authentification

L'inscription publique générale a été désactivée. L'inscription client publique crée une fiche `client`, hash le mot de passe avec `bcrypt`, crée un `user` avec le rôle forcé `client`, puis retourne un JWT.

À la connexion, le service recherche l'email, récupère le mot de passe masqué avec `select("+mot_de_passe")`, utilise `bcrypt.compare()`, puis appelle `jwt.sign()`.

Le JWT contient `id`, `id_client` et `role`. `authMiddleware` lit le header `Authorization: Bearer ...`, vérifie le token et place son contenu dans `req.user`.

7. Autorisation

Rôle	Permissions principales
admin	Gestion des utilisateurs, clients, plats, catégories, commandes et factures.
serveur	Clients, consultation du menu et création/modification des commandes.
caissier	Consultation clients/menu, commandes et gestion des factures hors suppression.
client	Son profil, ses commandes et ses factures uniquement.

`roleMiddleware` contrôle le rôle du JWT. `ownerMiddleware` limite les profils au propriétaire ou à l'admin.

8. CRUD

Entité	Créer	Lire	Mettre à jour	Supprimer
--------	-------	------	---------------	-----------

user	Oui	Oui	Oui	Oui
client	Oui	Oui	Oui	Oui
plat	Oui	Oui	Oui	Oui
categori	Oui	Oui	Oui	Oui
commande	Oui	Oui	Oui	Oui
facture	Oui	Oui	Oui	Oui

Les opérations existent pour chaque entité, mais sont restreintes par les middleware de rôle.

9. Flux principaux

Inscription client

```
POST /authentification/inscription-client
→ validation Yup → AuthController → AuthService
→ création Client → bcrypt → création User(client)
→ JWT → réponse JSON
```

Commande client

```
POST /commande/creation-client
→ JWT → rôle client → validation
→ id_user et id_client issus du JWT
→ Controller → Service → Repository → MongoDB
```

Consultation personnelle

```
GET /commande/mes-commandes ou GET /facture/mes-factures
→ JWT → rôle client → req.user.id_client
→ find({ id_client }) → réponse filtrée
```

10. Base de données

La base configurée est gestion-de-restaurant. La connexion utilise DB_URL depuis .env, avec une URL MongoDB locale comme valeur de secours. Les repositories utilisent Mongoose pour créer, lire, mettre à jour et supprimer les documents.

11. Sécurité et risques détectés

- JWT, bcrypt, Yup, rôles et propriétaire sont présents.
- Le secret JWT du fichier `.env` est une valeur d'exemple et doit être remplacé en production.
- `cors`, `helmet` et `morgan` sont installés mais non utilisés.
- Swagger est installé mais non configuré.
- Le rôle client n'est pas autorisé à lire `/plat` et `/categorie`, alors qu'il peut commander.
- La route PUT commande utilise `validationMiddleware` sans schéma.
- Plusieurs mises à jour n'utilisent pas de validation Yup.
- Les controllers renvoient souvent directement l'erreur avec `res.json(error)`, sans statut HTTP précis.
- Les mises à jour ne demandent pas explicitement `new: true` ni `runValidators: true`.
- Le total et les prix des commandes ne sont pas recalculés côté serveur.
- La création client + user n'est pas transactionnelle.
- `client.model.js` utilise `require` au lieu de `required` pour `matricule`.
- Deux anciens fichiers dans `facture` sont vides et ne sont pas utilisés.
- Aucun test automatisé n'est configuré.

12. Exemple : créer une commande client

Le client envoie uniquement les lignes de plats et le total à `POST /commande/creation-client` avec son JWT. Le middleware vérifie le token et le rôle. Le controller ignore les identifiants d'identité envoyés et utilise `req.user.id` et `req.user.id_client`. Le service appelle le repository, qui exécute `commandemodel.create()`. MongoDB enregistre ensuite la commande et l'API renvoie la réponse HTTP 201.

13. Résumé rapide

1. API de gestion de restaurant.
2. Backend Node.js et Express.
3. MongoDB avec Mongoose.
4. Gestion des clients, plats et catégories.
5. Gestion des commandes et factures.
6. Architecture routes/controllers/services/repositories/models.
7. JWT pour l'authentification.

8. bcrypt pour les mots de passe.
9. Rôles admin, serveur, caissier et client.
10. Le client crée son compte et ses commandes.
11. Le client voit seulement ses données personnelles.
12. Les routes métier sont protégées selon les rôles.
13. Yup valide les requêtes.
14. Des renforcements de sécurité restent à prévoir.

14. Diagramme final

```
CLIENT HTTP
  ↓
app.js / Routes Express
  ↓
authMiddleware → roleMiddleware → ownerMiddleware → validation Yup
  ↓
Controllers
  ↓
Services
  ↓
Repositories
  ↓
Models Mongoose
  ↓
MongoDB : gestion-de-restaurant
```